

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO THỰC TẬP

NGÀNH: CÔNG NGHỆ THÔNG TIN

ĐỀ TÀI: XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG
QUẢN LÝ TIẾN ĐỘ ĐỌC, CHIA SẺ VÀ ĐỀ XUẤT
SÁCH ỨNG DỤNG MLOPS VÀ DEVSECOPS

Cán bộ hướng dẫn: Nguyễn Quốc Thắng

Giảng viên đánh giá: ThS. Ngô Xuân
Trường

Sinh viên: Nguyễn Quang Dũng

Mã sinh viên: 23021497

Lớp: K68-CS3

Hà Nội, tháng 10 năm 2026

MỤC LỤC

LỜI CẢM ƠN	3
I. GIỚI THIỆU CHUNG	5
II. YÊU CẦU BÀI TOÁN	6
III. TÓM TẮT LÝ THUYẾT, GIẢI PHÁP, THUẬT TOÁN	7
IV. MÔ TẢ PHẦN MỀM CÀI ĐẶT	12
V. KẾT QUẢ ĐẠT ĐƯỢC, HƯỚNG PHÁT TRIỂN	15
TÀI LIỆU THAM KHẢO	17

LỜI CẢM ƠN

Trong quá trình thực tập và thực hiện báo cáo với đề tài này, em đã nhận được sự quan tâm, hướng dẫn và hỗ trợ quý báu từ nhà trường, đơn vị thực tập cùng các thầy cô, anh chị hướng dẫn.

Trước hết, em xin chân thành cảm ơn Ban lãnh đạo và các anh chị tại JITS Innovation Labs đã tạo điều kiện để em được tiếp cận môi trường làm việc thực tế, củng cố kiến thức chuyên môn và rèn luyện kỹ năng nghề nghiệp. Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc tới cán bộ hướng dẫn Nguyễn Quốc Thắng đã tận tình định hướng, góp ý và hỗ trợ em trong suốt quá trình thực hiện đề tài.

Em cũng xin trân trọng cảm ơn các thầy cô Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội đã trang bị cho em nền tảng kiến thức cần thiết. Em xin cảm ơn ThS. Ngô Xuân Trường đã dành thời gian đánh giá và góp ý cho báo cáo.

Mặc dù đã nỗ lực hoàn thiện, báo cáo khó tránh khỏi những thiếu sót do kinh nghiệm thực tế còn hạn chế. Em rất mong nhận được những ý kiến đóng góp để tiếp tục hoàn thiện kiến thức và kỹ năng trong thời gian tới.

Em xin trân trọng cảm ơn.

DANH MỤC HÌNH

Hình 1. Kiến trúc triển khai và luồng phụ thuộc chính của Philobiblus	9
Hình 2. Trang đăng nhập được chụp từ frontend Philobiblus chạy cục bộ	10
Hình 3. Pipeline snapshot, huấn luyện, đánh giá và phát hành model	11
Hình 4. Grafana ghi nhận backend trong một lần kiểm tra tải và HPA	14
Hình 5. Các lớp kiểm soát từ thay đổi mã nguồn đến workload GKE	15
Hình 6. Độ trễ và tỷ lệ lỗi tại các mốc tải chung trước và sau tối ưu	17

I. GIỚI THIỆU CHUNG

a. Giới thiệu Công ty TNHH JITS Innovation Labs

Công ty TNHH JITS Innovation Labs là doanh nghiệp hoạt động trong lĩnh vực phát triển phần mềm và cung cấp dịch vụ công nghệ thông tin tại Hà Nội. Công ty bắt đầu hoạt động từ tháng 6 năm 2016, có quy mô từ 11 đến 50 nhân sự, đặt trụ sở tại Hà Nội. Quy mô này tạo điều kiện để các nhóm kỹ thuật phối hợp trực tiếp, đồng thời yêu cầu mỗi thành viên chủ động trong quá trình nghiên cứu, phát triển và triển khai sản phẩm.

Lĩnh vực chuyên môn của JITS Innovation Labs là các giải pháp công nghệ cho ngân hàng và tổ chức tài chính, bao gồm nền tảng ngân hàng số, hệ thống điểm bán hàng, xử lý dữ liệu theo luồng và tích hợp hệ thống. Đội ngũ của công ty có kinh nghiệm triển khai, tùy biến hệ thống ngân hàng lõi cho khách hàng doanh nghiệp, hỗ trợ các ngân hàng và tổ chức tài chính vì mô cải tiến sản phẩm và thực hiện chuyển đổi số. Đặc thù của các hệ thống tài chính đòi hỏi phần mềm có tính ổn định, khả năng mở rộng, bảo mật và giám sát tốt; vì vậy, các phương pháp DevOps và DevSecOps có vai trò thiết thực trong hoạt động phát triển và vận hành sản phẩm.

Bên cạnh hoạt động phát triển sản phẩm, doanh nghiệp tổ chức chương trình thực tập theo hướng đào tạo gắn với thực hành. Tham gia vào kỳ thực tập năm nay, sinh viên chúng em được tiếp cận kiến thức theo lộ trình, hoàn thành các bài tập nhỏ cho từng chủ đề, lưu trữ sản phẩm trên hệ thống quản lý mã nguồn và trình bày kết quả định kỳ. Hình thức này giúp chúng em hình thành thói quen tự học, viết tài liệu, kiểm chứng kết quả và chịu trách nhiệm đối với sản phẩm đã thực hiện. Trong kỳ thực tập, em tham gia chương trình đào tạo DevOps và lựa chọn hướng chuyên sâu MLOps và Security để vận dụng kiến thức vào dự án tổng hợp.

Thông tin doanh nghiệp được đối chiếu từ trang chủ, trang LinkedIn và thông tin đăng ký doanh nghiệp công khai [1, 2, 3].

b. Giới thiệu công việc thực tập

Trong thời gian thực tập tại JITS Innovation Labs, em đảm nhiệm vị trí DevOps Intern. Em vận dụng kiến thức về Linux, Git, Docker, CI/CD, Kubernetes, Terraform, MLOps và DevSecOps để hoàn thành các bài thực hành và dự án tổng hợp. Quá trình này giúp em hình thành cách tiếp cận công việc từ tổ chức mã nguồn, xây dựng môi trường chạy đến kiểm tra và vận hành sản phẩm.

Từ các kiến thức nền tảng, em xây dựng và đóng gói các ứng dụng web nhiều thành phần, triển khai chúng trên Kubernetes, đồng thời áp dụng hạ tầng dưới dạng mã, tự động kiểm thử và cơ chế đồng bộ triển khai. Kết quả là các ứng dụng có thể chạy trên môi trường Kubernetes local, còn thay đổi mã nguồn được đưa qua quy trình tự động trước khi sử dụng.

Trên cơ sở đó, em tiếp tục mở rộng sang MLOps và bảo mật. Em xây dựng luồng phục vụ mô hình giả lập, áp dụng kiểm soát an toàn cho image và workload, đồng thời mô phỏng pipeline xử lý dữ liệu, huấn luyện, đánh giá và triển khai theo sự kiện. Các trường hợp không đáp ứng yêu cầu bảo mật được phát hiện trong quá trình kiểm thử; kết quả thực thi workflow và báo cáo phân tích rủi ro là những đầu ra trực tiếp của quá trình thực hành.

Những kinh nghiệm trên được tổng hợp trong sản phẩm cuối kỳ Philobiblus, một ứng dụng web hỗ trợ quản lý tiến độ đọc, nhận xét, chia sẻ và gợi ý sách. Sản phẩm kết hợp các nội dung đã thực hành để tổ chức mã nguồn, đóng gói ứng dụng, tự động kiểm thử và triển khai thử nghiệm. Qua đó, em có thể hoàn thiện một sản phẩm từ giai đoạn phát triển đến môi trường vận hành và đánh giá được các yêu cầu về tính ổn định, khả năng mở rộng và bảo mật.

c. Giới thiệu qua bài toán

Philobiblus không dừng ở các màn hình quản lý sách. Hệ thống phải giữ đúng quyền sở hữu dữ liệu, cho phép chia sẻ có kiểm soát và tích hợp gợi ý mà không làm gián đoạn chức năng chính. Vì vậy, giải pháp tách giao diện, backend nghiệp vụ và dịch vụ gợi ý để từng phần có thể được kiểm thử và vận hành độc lập.

II. YÊU CẦU BÀI TOÁN

a. Bối cảnh và mô tả bài toán

Thông tin về quá trình đọc thường bị phân tán giữa ghi chú, mạng xã hội và nhiều công cụ riêng lẻ. Người dùng vì thế khó theo dõi sách đang đọc, lịch sử hoàn thành và mục tiêu cá nhân.

Philobiblus tập trung các thao tác đăng ký, quản lý thư viện, cập nhật tiến độ, viết nhận xét, chia sẻ theo phạm vi và nhận gợi ý sách. Người dùng thông thường quản lý dữ liệu đọc và tương tác với nội dung công khai; quản trị viên xử lý dữ liệu cần kiểm soát.

b. Mục tiêu của hệ thống

Mục tiêu nghiệp vụ của Philobiblus là cung cấp một không gian thống nhất để người dùng quản lý việc đọc, ghi nhận đánh giá, chia sẻ có kiểm soát và khám phá sách mới. Các thao tác từ thêm sách, cập nhật tiến độ đến xem lại lịch sử phải được thực hiện trong cùng một tài khoản, với dữ liệu được gắn đúng chủ sở hữu.

Mục tiêu kỹ thuật là xây dựng một sản phẩm có thể triển khai lặp lại trên Google Kubernetes Engine (GKE), tách rõ giao diện, API, dữ liệu và dịch vụ gợi ý. Hệ thống phải cho phép tự động kiểm thử và phát hành, theo dõi được trạng thái workload, đồng thời quản lý model theo phiên bản thay vì chép thủ công artifact vào máy chủ.

c. Yêu cầu của hệ thống

Yêu cầu chức năng

Quản lý tài khoản và quyền truy cập. Người dùng có thể đăng ký, đăng nhập, cập nhật hồ sơ và đăng xuất; backend phải xác thực token, kiểm tra chủ sở hữu tài nguyên và áp dụng quyền quản trị ở phía máy chủ.

Quản lý thư viện và tiến độ đọc. Người dùng có thể tạo, tìm kiếm, chỉnh sửa và xóa sách thuộc quyền sở hữu; mỗi sách phải lưu được trạng thái dự định đọc, đang đọc hoặc đã hoàn thành, cùng tiến độ và lịch sử thay đổi.

Tương tác và chia sẻ. Người dùng có thể viết nhận xét, chấm điểm, theo dõi tài khoản khác và chia sẻ sách theo các mức public, private hoặc restricted; dữ liệu không được hiển thị vượt quá visibility đã chọn.

Gợi ý nội dung. Hệ thống phải trả về danh sách sách liên quan từ catalog hợp lệ, loại bỏ sách nguồn và các bản ghi không đủ metadata, đồng thời vẫn phục vụ được thư viện khi dịch vụ gợi ý tạm thời không sẵn sàng.

Quản trị vòng đời model. Hệ thống phải tạo snapshot dữ liệu, ghi nhận run và metric, kiểm tra chất lượng candidate rồi chỉ phát hành model đạt điều kiện; model đang phục vụ phải truy nguyên được về snapshot, mã nguồn và artifact tương ứng.

Yêu cầu phi chức năng

Bảo mật và phân quyền. Mật khẩu phải được băm, API phải kiểm tra JWT và quyền sở hữu, secret không đặt trong mã nguồn; các lớp Gateway, Cloud Armor, NetworkPolicy và securityContext phải hạn chế truy cập từ ngoài vào trong cluster.

Khả năng triển khai lặp lại. Hạ tầng GKE, Cloud SQL, mạng và quyền truy cập phải được mô tả bằng Terraform để có thể kiểm tra thay đổi, tái tạo môi trường và tránh cấu hình thủ công không truy vết được. Đây là lý do dự án áp dụng Infrastructure as Code.

Khả năng mở rộng và tự phục hồi. Các thành phần chạy dạng container phải được đóng gói thành Deployment hoặc Job trên Kubernetes, có readiness/liveness probe, giới hạn tài nguyên và cơ chế rollout; nhờ đó workload có thể tự khởi động lại và mở rộng theo nhu cầu.

Tính lặp lại của phát hành. Mỗi thay đổi phải đi qua CI/CD gồm kiểm thử, kiểm tra dependency, quét bảo mật, build image và triển khai bằng Helm với image digest xác định; việc phát hành không phụ thuộc vào thao tác thủ công trên cluster.

Quản lý vòng đời dữ liệu và model. Snapshot, pipeline, metric, artifact và model version phải được lưu vết bằng DVC và MLflow, cho phép tái hiện một lần huấn luyện và rollback khi candidate không đạt quality gate. Đây là yêu cầu cốt lõi dẫn đến việc áp dụng MLOps.

Quan sát và xử lý sự cố. Hệ thống phải xuất metric, log và trạng thái rollout để Cloud Monitoring, Cloud Logging và Managed Service for Prometheus có thể phát hiện lỗi, theo dõi tải và hỗ trợ đối chiếu nguyên nhân.

Khả năng kiểm thử và bảo trì. Backend, recommendation service, Helm chart và Terraform module phải được kiểm thử độc lập; cấu trúc module, schema và hợp đồng API cần đủ rõ để thay đổi một thành phần không làm hỏng các thành phần còn lại.

d. Phạm vi bài toán và trách nhiệm thực hiện

Philobiblus là dự án độc lập do em thực hiện từ phân tích yêu cầu, phát triển giao diện, API và cơ sở dữ liệu đến MLOps, kiểm thử, đóng gói và triển khai. Sản phẩm được đánh giá qua các luồng quản lý thư viện, tiến độ, nhận xét, chia sẻ, gợi ý và khả năng duy trì ranh giới quyền hạn.

Dự án không xử lý thanh toán, giao nhận, bản quyền nội dung số hoặc kho sách vật lý. Hệ gợi ý phục vụ catalog hiện có, chưa hướng tới huấn luyện quy mô thương mại.

III. TÓM TẮT LÝ THUYẾT, GIẢI PHÁP, THUẬT TOÁN

a. Các lý thuyết, giải pháp và thuật toán liên quan

Phần này chỉ giới thiệu những khái niệm trực tiếp chi phối cách thiết kế Philobiblus; chi tiết cấu hình được trình bày ở phần triển khai.

Các lý thuyết nền tảng

CI/CD. CI/CD tự động hóa chuỗi kiểm thử, đóng gói và phát hành, giúp mỗi thay đổi được kiểm tra trước khi đi vào môi trường chạy.

Infrastructure as Code. Infrastructure as Code biểu diễn hạ tầng bằng mã có phiên bản, từ đó tạo môi trường nhất quán và cho phép xem xét thay đổi trước khi áp dụng.

Containerization. Container đóng gói ứng dụng cùng dependency trong image bất biến, giảm khác biệt giữa môi trường phát triển, kiểm thử và production [4].

Điều phối workload. Kubernetes quản lý Pod, Deployment, Service, probe, rollout và autoscaling để workload có thể tự phục hồi và mở rộng theo chính sách đã khai báo [5].

MLOps. MLOps mở rộng quy trình DevOps cho dữ liệu và model, bao gồm versioning snapshot, tracking run, đánh giá candidate, promotion và rollback.

DevSecOps. DevSecOps đưa kiểm tra bảo mật vào cùng pipeline phát triển, thay vì chờ đến cuối chu kỳ mới quét image, dependency hoặc cấu hình.

Observability. Observability kết hợp metric, log và trạng thái trace để suy luận nguyên nhân sự cố từ dữ liệu vận hành thay vì chỉ dựa vào mã phản hồi.

Các giải pháp và công cụ áp dụng

Terraform. Terraform quản lý GKE, mạng, IAM và các tài nguyên Cloud SQL theo mô hình khai báo, giúp tái tạo hạ tầng mà không phải cấu hình từng bước trên giao diện.

Google Cloud và GKE. GKE Autopilot cung cấp cluster Kubernetes managed, còn Google Cloud cung cấp Gateway, Cloud Armor, Cloud SQL, GCS và các dịch vụ quan sát mà hệ thống cần khi triển khai thật.

Docker. Docker tạo image riêng cho frontend, backend, recommendation và các job huấn luyện để pipeline có thể kiểm tra và phát hành cùng một artifact.

Helm. Helm gom Deployment, Service, ConfigMap, Secret và policy thành chart có tham số, giúp một release được triển khai nhất quán trên cluster.

GitHub Actions. GitHub Actions thực thi các bước test, lint, scan, build, ký image và gọi quy trình triển khai khi mã nguồn thay đổi.

DVC. DVC liên kết snapshot dữ liệu với các stage prepare, train và evaluate để kết quả huấn luyện có thể tái hiện [6].

MLflow. MLflow lưu run, metric, parameter, artifact và model version, tạo điểm kiểm soát cho quyết định promotion hoặc rollback [7].

Cloud SQL và Redis. Cloud SQL lưu dữ liệu nghiệp vụ bền vững, còn Redis đảm nhiệm cache catalog và rate limit để giảm tải cho database.

Cloud Monitoring và Managed Service for Prometheus. Hai dịch vụ này tiếp nhận metric, log và cảnh báo từ workload GKE để theo dõi request rate, latency, lỗi và mức sử dụng tài nguyên [8].

Các thuật toán sử dụng

TF-IDF. TF-IDF biến metadata sách thành vector thưa, trong đó các từ xuất hiện nhiều trong một sách nhưng ít xuất hiện trong catalog có trọng số cao [9].

Cosine similarity. Cosine similarity đo góc giữa hai vector TF-IDF để xếp hạng các sách có nội dung gần với sách nguồn mà không phụ thuộc độ dài tuyệt đối của chuỗi.

Snapshot hash và quality gate. Hệ thống băm nội dung logic của snapshot, đối chiếu schema và mức suy giảm catalog trước khi cho phép candidate thay champion.

b. Cách giải quyết của sinh viên

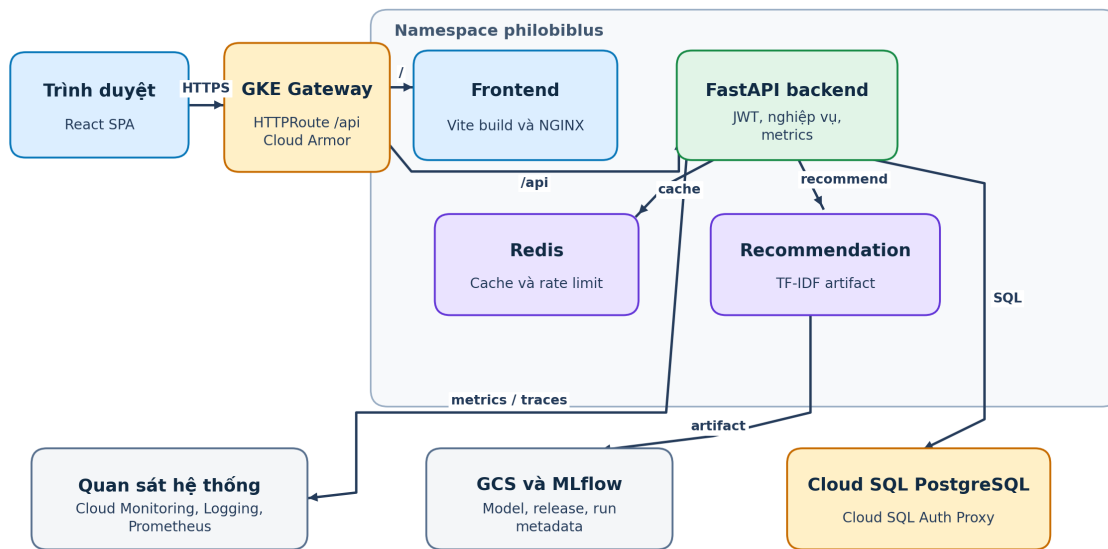
Em xây dựng frontend React, backend FastAPI và cơ sở dữ liệu PostgreSQL. Hệ thống cung cấp đăng ký, đăng nhập, hồ sơ, thư viện cá nhân, tiến độ đọc, nhận xét, chia sẻ và gợi ý sách.

Kiến trúc tổng thể và ranh giới trách nhiệm

React xử lý giao diện; FastAPI kiểm tra token, quyền sở hữu và nghiệp vụ; PostgreSQL lưu dữ liệu quan hệ. Frontend chỉ gọi API và không truy cập trực tiếp database hoặc model artifact.

Trên GKE, Gateway và HTTPRoute chỉ công khai tiền tố /api. Cloud Armor bảo vệ tầng biên; Service chuyển request tới backend Pod; NetworkPolicy giới hạn traffic giữa backend, Redis, recommendation và hệ thống thu metric.

Kiến trúc triển khai Philobiblus trên GKE



Hình 1: Kiến trúc triển khai và luồng phụ thuộc chính của Philobiblus

Backend kết nối Cloud SQL qua Cloud SQL Auth Proxy. Redis lưu cache catalog và trạng thái rate limit dùng chung. Recommendation service nạp artifact đã kiểm tra, trả danh sách xếp hạng và có cơ chế lỗi không làm gián đoạn catalog chính.

Mô hình dữ liệu và kiểm soát quyền truy cập

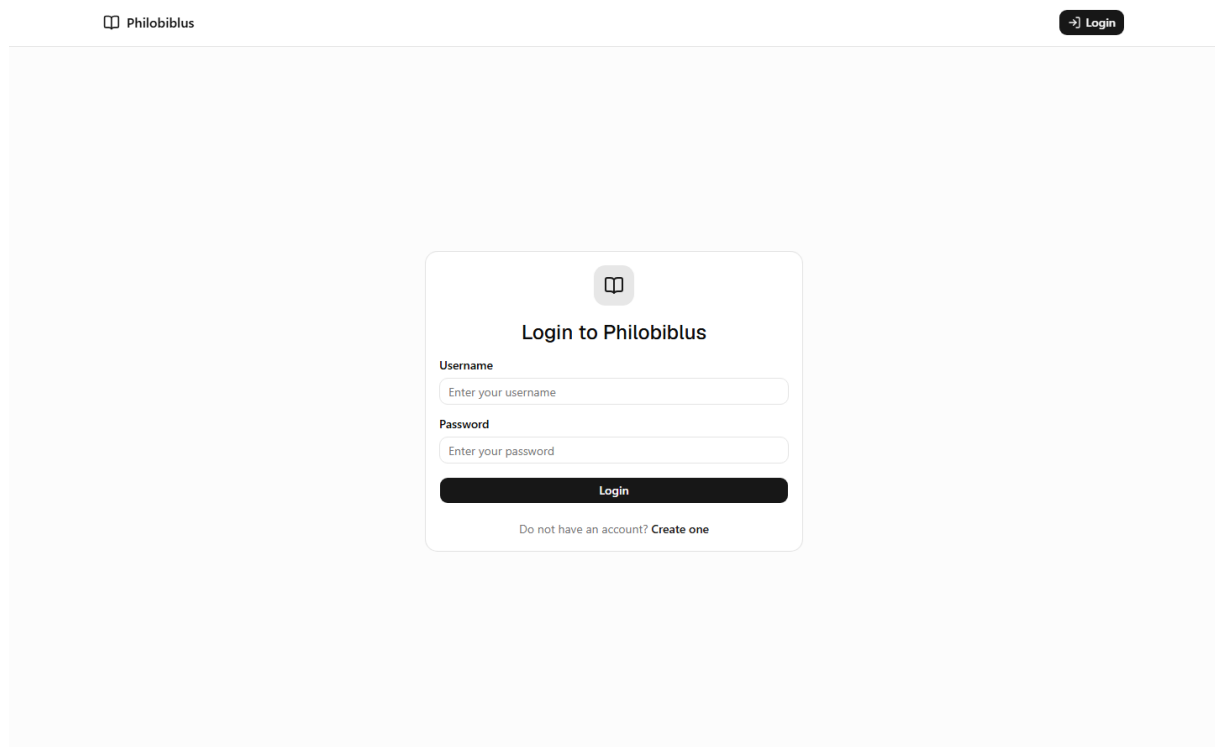
User và UserSettings lưu định danh, trạng thái và thiết lập. Book gắn với chủ sở hữu, tiến độ, metadata và mức public, private hoặc restricted. BookReadingProgress, ReadingHistory, Review, Follow và Friendship ghi nhận tiến độ riêng và tương tác xã hội mà không sửa dữ liệu của người khác.

Backend kiểm tra token cùng user_id của tài nguyên trước thao tác đọc, sửa hoặc xóa. Endpoint quản trị yêu cầu is_admin; truy vấn công khai lọc visibility trước khi trả dữ liệu. Việc ẩn nút ở frontend không được xem là biện pháp phân quyền.

Pipeline kết hợp tên, tác giả, thể loại và nhãn của sách public thành đặc trưng TF-IDF. Dịch vụ gợi ý xếp hạng tối đa năm sách theo cosine similarity, loại sách nguồn và các mục không hợp lệ trước khi trả kết quả.

Giao diện và luồng xác thực

Frontend tách route công khai, route cần đăng nhập và route quản trị. AuthContext lưu access token; ProtectedRoute và AdminRoute kiểm soát trải nghiệm, còn backend đưa ra quyết định phân quyền cuối cùng.



Hình 2: Trang đăng nhập được chụp từ frontend Philobiblus chạy cục bộ

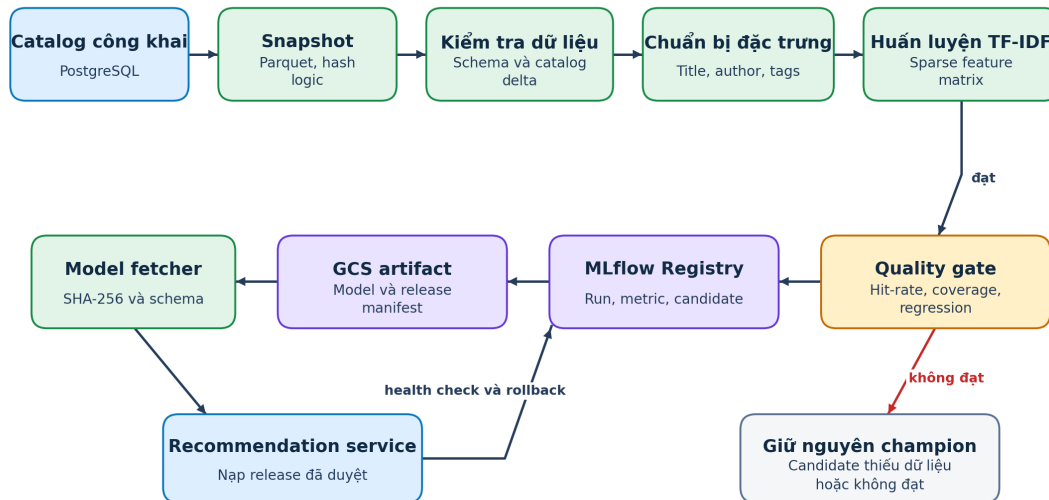
Form đăng nhập gửi username và password tới `/api/auth/login`, khóa nút khi chờ phản hồi và hiển thị lỗi tại chỗ. Sau xác thực, người dùng truy cập thư viện, thống kê, trang xã hội và cài đặt; quản trị viên có thêm các màn hình quản lý.

Hệ thống hiện được triển khai trên GKE bằng Terraform và Helm. Pipeline DVC tạo dữ liệu huấn luyện; MLflow trên cluster ghi run và model version; artifact bất biến được lưu trên GCS. Pod-Monitoring gửi metric tới Google Managed Service for Prometheus, còn Cloud Monitoring và Cloud Logging phục vụ quan sát.

Vòng đời dữ liệu và mô hình gợi ý

Pipeline không đọc trực tiếp một tập dữ liệu đang thay đổi trong suốt quá trình huấn luyện. `snapshot_catalog.py` mở transaction repeatable-read, chọn sách public, xuất Parquet và tính `snapshot_id` từ dữ liệu logic. `validate_snapshot.py` kiểm tra schema, ID, metadata và mức suy giảm catalog so với champion. DVC liên kết snapshot với bước prepare, train và evaluation. Lần xác minh dùng 100 sách và tạo 2.030 đặc trưng; candidate thiếu cohort tương tác được ghi nhận là `insufficient_data` nên không được phép thay model đang phục vụ.

Vòng đời dữ liệu và mô hình gợi ý



Hình 3: Pipeline snapshot, huấn luyện, đánh giá và phát hành model

MLflow lưu tham số, metric, run và model version; GCS giữ snapshot, model cùng release manifest bất biến. Trước khi recommendation service khởi động, model-fetcher tải đúng release, kiểm tra SHA-256 và schema rồi mới công bố file trong volume dùng chung. Promotion cập nhật release pointer bằng resource version, chờ Deployment rollout và gọi health endpoint để đối chiếu model_version. Nếu rollout hoặc health check thất bại, pipeline khôi phục pointer và annotation trước đó. Nhờ vậy, artifact sai định dạng hoặc sai phiên bản không được đưa vào hệ thống.

c. Liên hệ và so sánh với các cách đã nêu

TF-IDF không cần ma trận tương tác dày và xử lý tốt catalog mới, đổi lại phụ thuộc chất lượng metadata. Khi lịch sử đọc đủ lớn, hệ thống có thể kết hợp điểm nội dung với hành vi cộng đồng.

Các đánh đổi kỹ thuật trong giải pháp

Redis cache catalog trong 30 giây và được invalidation khi dữ liệu thay đổi. Nếu Redis lỗi, backend bỏ qua cache để tiếp tục phục vụ, chấp nhận tăng tải tạm thời cho Cloud SQL.

Mỗi backend Pod dùng pool 3 kết nối và tối đa 2 kết nối vượt mức, timeout 10 giây. Giới hạn này được tính cùng HPA để tránh tổng số connection vượt khả năng của Cloud SQL db-f1-micro.

TF-IDF dễ giải thích nhưng chưa khai thác sâu hành vi cộng đồng. GKE chuẩn hóa rollout, probe và autoscaling, đổi lại yêu cầu quản lý chặt tài nguyên, quyền cloud và chi phí.

IV. Mô tả phần mềm cài đặt (nếu có)

a. Không cần mô tả cách cài đặt các ứng dụng phụ

Người dùng truy cập Philobiblus bằng trình duyệt và không phải cài phần mềm desktop. GKE, Helm, Terraform, Cloud SQL và các dịch vụ quan sát chỉ phục vụ triển khai, vận hành.

b. Mô tả ngắn gọn cách cài đặt phần mềm được phát triển

Phiên bản được mô tả trong báo cáo chạy trên GKE Autopilot. Frontend tĩnh gọi API qua Gateway; backend và recommendation chạy trong namespace philobiblus, Redis cung cấp cache, còn dữ liệu nghiệp vụ nằm trên Cloud SQL.

Quy trình triển khai trên GKE

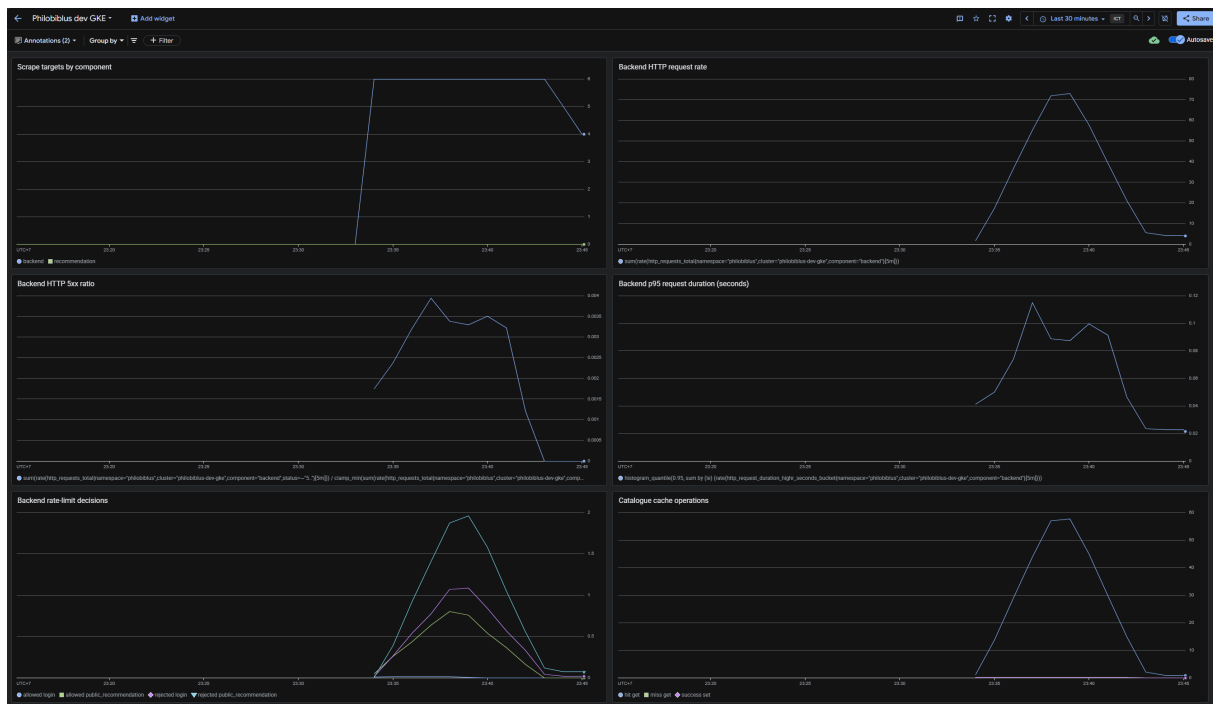
Hạ tầng được tách thành các Terraform state gke-platform, gke-app, gke-mlops và gke-observability. gke-platform quản lý GKE Autopilot, mạng, Gateway và Cloud Armor; gke-app quản lý namespace, Workload Identity, Cloud SQL, Helm release và cấu hình ứng dụng; gke-mlops quản lý bucket versioned, IAM theo prefix, MLflow và pipeline retrain; gke-observability quản lý Cloud Monitoring dashboard cùng alert policy. Tách state riêng giúp việc thay đổi dashboard hoặc model pipeline mà không lập kế hoạch lại toàn bộ cluster.

Image được tham chiếu bằng digest bất biến để cùng một release luôn trở tới cùng nội dung. Helm tạo Deployment, Service, Redis, HTTPRoute, GCPBackendPolicy, NetworkPolicy, PodMonitoring, readiness/liveness probe và HPA. Backend dùng tối thiểu 2, tối đa 6 Pod trong đợt đo tải; probe chỉ đưa Pod vào Service khi sẵn sàng. Workload nhận quyền Google Cloud qua Workload Identity, trong khi secret được cung cấp ngoài chart thay vì ghi trực tiếp trong repository.

Release đi qua kiểm thử, build image, lấy digest, validate Terraform, render Helm và chờ roll-out. Metric nội bộ được PodMonitoring thu vào Google Managed Service for Prometheus; Cloud Monitoring và Cloud Logging là giao diện quan sát của GKE.

Quan sát hệ thống và xử lý sự cố

FastAPI xuất /metrics trong mạng nội bộ. PodMonitoring thu request rate, latency, mã phản hồi, rate limit và cache operation; Cloud Monitoring kết hợp chúng với CPU, memory, restart và số replica. Log stdout/stderr được Cloud Logging thu theo cluster, namespace và container.



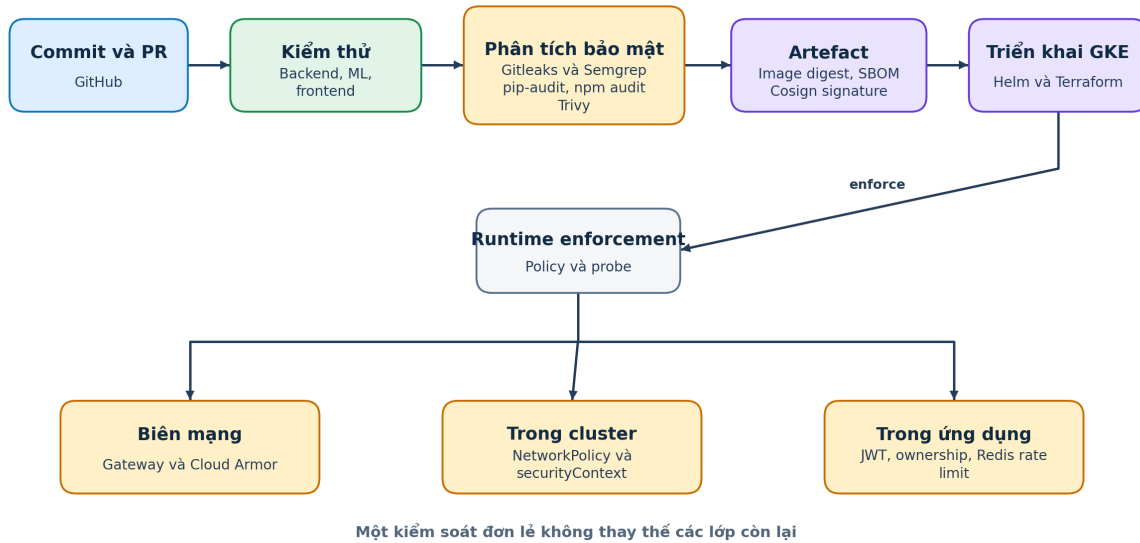
Hình 4: Grafana ghi nhận backend trong một lần kiểm tra tải và HPA

Dashboard ghi lại diễn biến target, request rate và p95 trong lần tạo tải. Khi phân tích sự cố cần đối chiếu khoảng thời gian, metric tài nguyên và log, không kết luận chỉ từ một panel.

Chuỗi kiểm soát DevSecOps

CI sử dụng Gitleaks, Semgrep, pip-audit, npm audit và Trivy; bước phát hành tạo SBOM và ký image bằng Cosign trước khi triển khai. Cảnh báo quét được phân loại theo khả năng khai thác thay vì xử lý như nhau.

Chuỗi kiểm soát DevSecOps từ commit đến GKE



Hình 5: Các lớp kiểm soát từ thay đổi mã nguồn đến workload GKE

Sau khi workload hoạt động, Cloud Armor, Gateway, NetworkPolicy, securityContext và Redis rate limiter bảo vệ các ranh giới khác nhau. HTTPRoute chỉ công khai /api; /metrics, /docs, /openapi.json và /health trả 404 qua Gateway nhưng vẫn dùng được cho probe và thu metric nội bộ. Request thử SQL injection và XSS khớp rule CRS 4.22 của Cloud Armor ở chế độ preview. Chuỗi đăng nhập sai vượt ngưỡng bị rate limiter ứng dụng trả 429, cho thấy lớp bảo vệ trong backend vẫn hoạt động khi rule biên chưa chuyển sang Enforce.

V. Kết quả đạt được, hướng phát triển

a. Kỹ năng và kiến thức thu thập được

Trong kỳ thực tập, em theo dõi toàn bộ hệ thống từ mã nguồn, kiểm thử và image đến Terraform, Helm, GKE và quan sát vận hành. Em hiểu rõ hơn cách Service, probe, resource, secret, policy mạng và rollout phối hợp trong một workload thực tế.

Với MLOps và DevSecOps, em thực hành quản lý snapshot, artifact, quality gate, JWT, quyền sở hữu, Cloud Armor, NetworkPolicy và kiểm tra chuỗi cung ứng phần mềm.

Kết quả kỹ thuật đã kiểm chứng

Đợt này có tổng cộng 61 test backend, 8 test recommendation, frontend production build, lint/render hai Helm chart và terraform validate cho các module GKE. Các kiểm tra này bao phủ logic và cấu hình, chưa thay thế kiểm thử người dùng cuối.

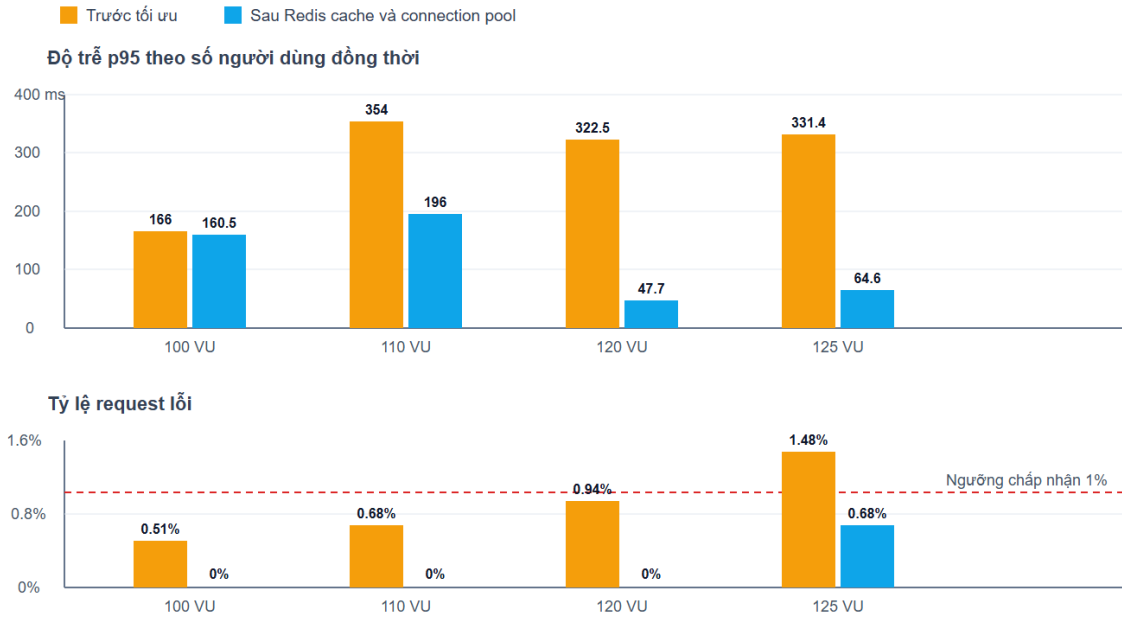
Ngày 24 tháng 9 năm 2026, pipeline trên project grace-enhanced và cluster philobiblus-dev-gke tạo snapshot từ 100 sách, ghi MLflow run và đăng ký model version 1. Candidate insufficient_data không được promote vì thiếu hồ sơ tương tác.

Release hardening triển khai 2 backend Pod, Redis, GCPBackendPolicy và ba NetworkPolicy. Các endpoint vận hành vẫn dùng nội bộ nhưng bị chặn qua Gateway; không ghi nhận OOMKilled, redis_error hoặc cạn connection trong lần xác minh.

b. Hướng phát triển tiếp theo để hoàn thiện giải pháp

Lần kiểm tra đầu đạt 120 người dùng đồng thời với 23,4 request/giây, p95 322,5 ms và 0,94 phần trăm lỗi; tại 125 người dùng, lỗi tăng lên 1,48 phần trăm. Sau khi thêm Redis cache TTL 30 giây và giới hạn pool 3 + overflow 2 cho mỗi Pod, mốc 150 người dùng đạt 29,6 request/giây, p95 138,6 ms và không có request lỗi.

So sánh tải trước và sau tối ưu ngày 24 tháng 9 năm 2026



Hình 6: Độ trễ và tỷ lệ lỗi tại các mốc tải chung trước và sau tối ưu

150 người dùng là mốc cao nhất đã kiểm tra và recommendation service được tắt để cô lập backend cùng database. Mức vận hành đề xuất là 120 đến 130 người dùng đồng thời, khoảng 24 request/giây, trước khi thực hiện lần đo đầy đủ hơn.

Cloud Armor vẫn ở chế độ preview để theo dõi false positive. Bước tiếp theo là xác minh chữ ký image, trace, PolicyReport và notification trước khi chuyển từng rule sang Enforce.

Hướng phát triển ưu tiên thu thập đủ hồ sơ độc cho holdout evaluation, sau đó thử mô hình lai và canary với hai release stable, candidate độc lập.

TÀI LIỆU THAM KHẢO

- [1] JITS Innovation Labs. (2026) Jits innovation labs. [Online]. Available: <https://jitsinnovationlabs.com/>
- [2] ——. (2026) Jits innovation labs trên linkedin. [Online]. Available: <https://www.linkedin.com/company/jits-innovation-labs>
- [3] Cổng thông tin quốc gia về đăng ký doanh nghiệp. (2026) Thông tin đăng ký doanh nghiệp công khai. [Online]. Available: <https://dangkykinhdoanh.gov.vn/>
- [4] Docker, Inc. (2026) What is a container? [Online]. Available: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>
- [5] The Kubernetes Authors. (2026) Deployments. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- [6] Iterative, Inc. (2026) Dvc command reference. [Online]. Available: <https://dvc.org/doc/command-reference>
- [7] MLflow Authors. (2026) Mlflow tracking. [Online]. Available: <https://mlflow.org/docs/latest/ml/tracking>
- [8] Prometheus Authors. (2026) Metric types. [Online]. Available: https://prometheus.io/docs/concepts/metric_types/
- [9] Scikit-learn Developers. (2026) Tfidfvectorizer and cosine similarity. [Online]. Available: <https://scikit-learn.org/stable/>

JITS INNOVATION LABS

Ý kiến đánh giá:

Hà Nội, ngày tháng năm 20 .

Người hướng dẫn

(Ký, ghi rõ họ tên & dấu công ty)



Ý kiến đánh giá:

Điểm số:

Điểm chữ:

Hà Nội, ngày tháng năm 20 .

Giảng viên đánh giá

(Ký, ghi rõ họ tên)